



Unparalleled Source

Vendor Management & Procurement Solutions

6 East Golf Club Lane, Paoli, PA 19301 | 484-886-0286 | 267-206-8170
unps@unparalleledsource.com | unparalleledsource.com
541611 • 541618 • 423430 • 541519 • 541512 • 541614 • 561210 • 484110 • 493110

CANNAGRAPH

MASTER BUILD PROMPT

& ACCEPTANCE QUALITY GATE

CLIENT HANDOFF

Prepared for Gregory Michael Donzuso

AI Consulting performed by Ernest Bey

Unparalleled Source

September 8, 2026

Database-first execution. Evidence integrity. Continuous discovery. Social participation. Acceptance before expansion.

Client Handoff Purpose

This handoff defines the build contract for Cannagraph: the technical architecture, product rules, implementation order, and acceptance criteria that should govern every coding session. The prompt is intentionally strict so a coding agent cannot trade provenance, database integrity, identity resolution, security, or maintainability for short-term speed.

Operating Rule

Load the master prompt once. Then instruct the coding agent to implement one slice only, for example: "Implement Slice 1 only. Stop after running the acceptance quality gate." Do not issue a broad "build Cannagraph" instruction.

Cannagraph Master Build Prompt

You are building Cannagraph, a premium, evidence-aware cannabis genetics research platform.

Cannagraph is not a simple strain directory, static encyclopedia, or generic social network.

It is a continuously evolving, source-backed cannabis genetics knowledge network designed to discover, research, catalog, cross-reference, deduplicate, preserve conflicting histories, and organize cultivars, traditional populations, breeder lines, seed lines, clone-only cuts, phenotypes, accessions, grows, batches, breeders, claims, evidence, sensory observations, chemistry measurements, lineage relationships, community contributions, grow journals, organizations, and research activity.

Your job is to implement the application incrementally without weakening the architecture for speed.

Core Stack

- Next.js App Router
 - TypeScript with strict mode
 - Supabase
 - PostgreSQL as the canonical database
 - Supabase Auth
 - Supabase Storage
 - Supabase Queues for durable research jobs
 - Supabase Cron only for scheduling/enqueuing work
 - Tailwind CSS
 - React Flow / @xyflow/react for graph exploration
 - Stripe for Cannagraph Pro subscriptions
 - PostgreSQL full-text search first
 - pgvector only where semantic evidence retrieval is actually needed
- Do not introduce a separate graph database, Elasticsearch, Redis, Kafka, or microservice architecture unless a measured production bottleneck justifies it.

Product Experience

Cannagraph must feel:

- premium
- scientific

- archival
- highly credible
- data-rich
- fast
- calm
- visually impressive

Avoid stereotypical cannabis branding.

Do not build a dispensary-looking interface.

Avoid:

- neon green themes
- cannabis leaf motifs
- novelty typography
- ecommerce strain cards
- excessive gradients
- noisy dashboards
- generic AI-generated SaaS styling

Prefer:

- strong typography
- warm neutral surfaces
- exceptional spacing
- archival-document treatments
- subtle evidence status colors
- detailed tables
- powerful graph visualization
- polished search
- research-terminal density where appropriate
- highly legible mobile layouts

Public browsing must require no login.

Authentication should appear only when the user attempts to save, follow, collect, receive alerts, or use account-specific functionality.

Do not implement the feed as a simple reverse-chronological SELECT from posts. Use structured activity events so the network remains genetics-aware.

The home feed must combine social activity with meaningful research events such as new sources, accepted claims, disputed lineage, grow updates, lab results, new descendants, verification events, and research-completeness changes.

Users must eventually be able to follow people and research objects, including cultivars, cuts, breeders, organizations, traditional populations, grow journals, collections, and research topics.

Genetics-Aware Social Experience

Never allow a crawler, LLM, API import, breeder account, grower account, or community post to directly overwrite canonical research truth.

Required flow: discover -> capture source -> identify candidate -> resolve identity -> create claims/evidence -> review -> update canonical graph -> emit activity event.

New information enters as a candidate, observation, claim, submission, or source artifact before it becomes canonical truth.

Cannagraph must be designed to evolve as new strains, cultivars, evidence, observations, and sources become available.

Continuous Evolution Rule

Community participation feeds the research system. It does not bypass or replace the research system.

The social layer supports profiles, organizations, follows, feeds, posts, grow journals, comments, collections, reactions, research discussions, breeder/grower verification, and structured community submissions.

3. Social and community layer

The discovery layer continually finds new cultivars, aliases, breeders, cuts, phenotypes, catalogs, lineage theories, sources, lab results, grow observations, and related entities from permitted public sources and community submissions.

2. Continuous discovery layer

The canonical layer stores entities, lineage, claims, evidence, sources, grows, batches, chemistry, sensory observations, and historical research state.

1. Canonical knowledge layer

Cannagraph must operate as three connected systems, not as a static database with social features bolted on later.

Living Network Product Model

User Types

Authorization roles:

- user
- researcher
- admin

Billing is independent from authorization.

Subscription levels:

- free
- pro

Never make pro equivalent to admin or researcher.

Architectural Rules

These rules are non-negotiable.

1. PostgreSQL is canonical

- All canonical domain state lives in PostgreSQL.
- Do not treat generated prose, vector embeddings, crawler documents, external APIs, or search indexes as canonical truth.

2. Names are not identities

- Never identify or merge a cultivar solely by its name.

- Every biological/commercial entity gets its own immutable internal ID.
- Examples: CNG-CUL-00000001; CNG-CUT-00000001; CNG-BRE-00000001; CNG-POP-00000001.
- A PostgreSQL UUID remains the relational primary key.

3. Important information is claim-based

- Do not store important historical/genetic information only as prose.
- Use structured claims with evidence.
- Every substantial claim must be traceable to one or more source documents.

4. Never fabricate missing information

- Unknown is valid.
- Use NULL, unknown status, unresolved status, or candidate status.
- Never infer unsupported parent direction, breeder, origin, release year, clone-only status, chemistry, sensory attributes, or identity.

5. Preserve conflicts

- Competing lineage and historical theories must coexist.
- Never silently replace an alternative theory with the currently preferred theory.

6. LLM output is not evidence

- An LLM may parse documents, extract candidates, normalize wording, identify possible relationships, and suggest duplicate candidates.
- It may not independently establish a verified fact.
- Every verified or supported assertion requires underlying source evidence.

7. Chemistry is batch-specific

- Never assign one THC or terpene profile directly as an immutable cultivar property.
- Store measurements against cultivar/cut → grow → batch → laboratory test → analyte measurement.
- Cultivar-level chemistry is calculated from qualifying measurements.

8. Sensory reports remain scoped

- Separate breeder-reported, community-reported, reviewer-reported, grow-specific, and batch-specific observations.
- Do not convert one sensory report into a universal cultivar property.

9. No uncertain automatic merges

- Duplicate detection generates candidates.
- Human approval is required for uncertain identity merges.
- Merges must remain auditable and reversible.

10. Respect source restrictions

- Do not bypass login requirements, CAPTCHAs, paywalls, robots restrictions, anti-bot protections, API restrictions, or authentication barriers.
- A blocked source becomes a manual research source.

11. Community content cannot directly become canonical truth

- Community posts, journals, comments, submissions, reactions, follower counts, and reputation signals may inform research but cannot directly overwrite canonical facts.
- Accepted community contributions must pass through provenance, identity, evidence, and review workflows.

12. The database is intentionally extensible

- New entity types and observations should enter through additive migrations and stable relationship patterns.
- Unknown or emerging fields may begin in bounded metadata, but important repeated research concepts must graduate into normalized canonical columns or tables.

13. Social identity is separate from research authority

- Verified breeder/grower identity, subscription level, researcher authorization, admin authorization, and reputation are separate concepts.
- A verified breeder can publish an official statement without gaining authority to suppress conflicting historical evidence.

14. Research activity must be feedable

- Important canonical and community events should emit structured activity events so changes can power feeds, notifications, timelines, and audit history without scraping UI state.

Canonical Core Model

The initial database must include, at minimum:

- entities
- entity_names
- cultivars
- breeders
- cuts
- phenotypes
- traditional_populations
- sources
- source_documents
- claims
- claim_evidence
- lineage_relationships
- lineage_evidence
- hypotheses
- hypothesis_claims
- identity_candidates
- entity_merges
- source_registry
- discovery_queue
- research_jobs
- research_events
- organizations

- organization_members
- organization_entity_links
- verification_requests
- verification_evidence
- verification_events
- discovery_candidates
- community_submissions
- submission_entity_links
- submission_evidence
- activity_events

Later slices add:

- sensory_descriptors
- sensory_observations
- sensory_observation_descriptors
- grows
- batches
- laboratories
- lab_tests
- analytes
- lab_test_results
- search_documents
- profiles
- subscriptions
- entitlements
- usage_counters
 - posts
 - post_revisions
 - post_entity_links
 - comments
 - comment_revisions
- follows
- entity_follows
- collections
- collection_items
- grow_journals
- grow_journal_entries
- grow_journal_events
- grow_journal_media
- claim_challenges
- reports
- moderation_events

- content_status_history

Do not replace normalized tables with a giant JSONB document.

JSONB may be used for flexible metadata where the data is not yet important enough to deserve canonical columns.

Grow journals and profile data require privacy controls. Support private, unlisted, followers, organization, researchers, and public visibility where appropriate. Exact physical grow locations must never be required.

Moderation must distinguish content quality, research validity, community policy, identity disputes, copyright/source issues, spam, and abuse. Research disagreement is not automatically misconduct.

Do not equate likes, followers, or popularity with truth. Reputation should reflect useful contribution signals such as accepted evidence, accepted corrections, verified identity, citations, research contributions, and contribution longevity.

Reputation, Moderation, and Privacy

Activity events should cover canonical and social changes including cultivar_created, claim_added, claim_disputed, source_added, lineage_added, lineage_changed, grow_started, grow_updated, harvest_completed, lab_result_added, photo_added, collection_updated, identity_verified, and submission_accepted.

Collections should support private, unlisted, public, and collaborative visibility and may contain canonical entities, journals, or other supported research objects.

Posts should be linkable to canonical entities through post_entity_links so discussion can reference a cultivar, breeder, cut, grow, claim, source, or organization without duplicating those records.

Social Objects and Activity Events

Track source relationships such as original, derived, quoted, syndicated, copied, and unknown. Confidence calculations must account for source dependence.

Repeated copies of the same originating claim must not count as independent confirmation.

Source Dependence and Evidence Inflation

Accepted submissions create or support candidate claims/evidence. They do not directly mutate canonical records.

Submission lifecycle: draft -> submitted -> under_review -> needs_information -> accepted/partially_accepted/rejected/withdrawn/superseded.

Community submissions may propose missing cultivars, aliases, breeder attribution, historical accounts, lineage theories, corrections, photos, catalog scans, seed packs, breeder statements, grow observations, sensory observations, laboratory results, and source documents.

Important journal information should use structured events such as germination, transplant, vegetative start, training, flower start, pollination, environment change, feeding, pest event, disease event, phenotypic observation, harvest, drying, curing, lab submission, lab result, and sensory observation.

Grow journals are structured research-social objects tied to canonical cultivars/cuts, grows, batches, observers, organizations, and optional lab tests.

Grow Journals and Community Submissions

Verification is independent from billing and authorization. Pro does not mean verified. Verified does not mean researcher or admin.

Identity verification states should support unverified, pending, verified, disputed, and revoked, with auditable verification evidence and events.

Do not assume organization = breeder. A breeder may be an individual, an organization, or historically associated with multiple organizations.

Organizations are first-class entities representing breeders, seed companies, nurseries, cultivators, farms, laboratories, archives, publications, preservation groups, and research groups.

Organizations and Verified Identities

Recursive discovery may create additional candidates, but loop prevention, source/domain budgets, deduplication, and idempotent processing are mandatory.

Required lifecycle: candidate -> research -> evidence -> identity resolution -> canonical entity or rejected/unresolved candidate.

A candidate should preserve candidate_type, proposed_name, source_document_id, source_url, discovered_at, discovered_by, candidate_status, possible_entity_id, confidence, review_priority, and flexible metadata.

Use discovery_candidates as the controlled intake boundary for newly found cultivars, cuts, phenotypes, breeders, organizations, aliases, lineage relationships, sources, and catalogs.

Discovery Candidate Model

Database Migration Discipline

All database changes must be migration-driven.

Never manually create production tables without checked-in migrations.

Each migration should be:

- small
- clearly named
- deterministic
- safe to apply in sequence
- reversible when practical

Before changing an existing table:

1. inspect existing migrations
2. inspect dependent code
3. inspect constraints
4. inspect indexes
5. inspect RLS
6. create an additive migration whenever practical

Never reset production data to make development easier.

RLS and Security

RLS is mandatory.

Public users may read published public knowledge.

Normal users may only modify their own profile, follows, collections, saved searches, and account preferences.

Researchers may operate approved research workflows.

Admins receive administrative privileges.

Service-role operations remain server-only.

Never expose a Supabase service-role key to browser code.

Never trust a client-provided role or subscription level.

Organization membership and roles must be enforced by server-side authorization and RLS, not client state.

Potential organization roles: owner, admin, editor, contributor, member.

Users may edit their own community content subject to moderation and archival rules; they may not directly edit canonical research records.

Verification changes, moderation actions, organization ownership changes, submission acceptance, claim challenges, and canonical merges must remain auditable.

Build Order

Slice 0 — Foundation

- Next.js repository
- TypeScript strict mode
- Supabase local environment
- environment validation
- CI
- deployment setup
- route groups
- design-system primitives

Slice 1 — Canonical database

- core schema
- indexes
- constraints
- RLS
- migrations
- seed data

Slice 2 — Premium public shell

- /
- /search
- /cultivars
- /cultivars/[slug]
- /breeders/[slug]
- /sources/[id]
- Use the real database.
- Do not mock the final data architecture with temporary hard-coded JSON.

Slice 3 — Cultivar research page

- identity
- aliases
- research completeness
- history
- lineage summary

- evidence overview
- breeder
- descendants
- sources
- disputed claims

Slice 4 — Lineage graph

- interactive ancestor/descendant exploration
- Load graphs incrementally.
- Never render an uncontrolled entire graph.

Slice 5 — Authentication

- Anonymous browsing remains open.
- Account required only for personal features.

Slice 5A — Community identity foundation

- profiles, organizations, organization memberships, organization roles, public profile foundation, and verified identity workflow
- Keep verification independent from Pro, researcher, and admin status.

Slice 6 — Cannagraph Pro

- Implement Stripe billing and centralized entitlements.

Slice 7 — Research workstation

- Deliver researcher/admin queues and evidence review workflows.

Slice 7A — Community research submissions

- structured submissions, entity linkage, evidence submission, claim challenges, moderation queue, and researcher review
- Community submissions must never directly overwrite canonical records.

Slice 8 — Identity resolution

- Implement deterministic duplicate candidate scoring.
- Never auto-merge ambiguous entities.

Slice 9 — Source registry

- Implement source policies, schedules, rate limits, fetch permissions and source classification.

Slice 10 — Research jobs

- Implement durable queue processing.
- Research jobs must be idempotent.

Slice 11 — Scheduling

- Cron only discovers due work and enqueues jobs.
- Cron does not perform long-running research itself.

Slice 12 — First ingestion adapter

- Build one high-quality permitted source adapter.
- Prove end-to-end ingestion before adding more adapters.

Slice 13 — Recursive discovery

- New parents, breeders, aliases, catalogs and related entities become discovery candidates.
- Implement loop prevention and budgets.

Slice 14 — Search

- Implement PostgreSQL search projections and advanced structured filters.

Slice 15 — Sensory

- Implement normalized ontology plus source-scoped observations.

Slice 16 — Chemistry

- Implement batch-specific laboratory measurements.

Slice 16A — Grow journals

- structured grow journals, journal events, media, environmental observations, cultivar/cut linkage, batch linkage, sensory linkage, and lab-test linkage

Slice 17 — Research completeness

- Implement versioned quality/completeness scoring.

Slice 18 — Research Assistant

- Structured database retrieval comes before semantic retrieval.
- Every significant answer must be traceable to evidence.

Slice 18A — Social discovery and feeds

- posts, entity-linked discussion, follows, collections, activity events, notifications, and genetics-aware feeds
- Feed ranking must never change canonical truth or evidence status.

Slice 19 — Research campaigns

- Begin foundational cannabis genetics discovery only after ingestion and provenance systems are stable.

Slice 20 — Scale

- Only add specialized infrastructure after profiling demonstrates a real bottleneck.

Slice 20A — Public API readiness

- Stabilize public IDs, entity typing, provenance exposure, disputed-state representation, API clients/keys, usage accounting, rate limits, and webhooks only when the canonical model is ready.

Coding Rules

Before implementing a slice:

1. inspect all relevant existing code
2. inspect current migrations
3. identify dependencies

4. state the exact files you intend to modify
5. implement the smallest complete vertical slice
6. run tests
7. verify database migrations
8. verify responsive UI
9. verify permissions
10. verify acceptance criteria

Do not refactor unrelated code.

Do not silently rename domain concepts.

Do not create duplicate architectural systems because an existing abstraction is inconvenient.

Do not leave placeholder implementations presented as complete.

Do not use TODO to conceal missing acceptance-critical behavior.

Code Quality

Require:

- strict TypeScript
- explicit domain types
- server/client boundaries kept intentional
- Zod or equivalent boundary validation
- no any except documented unavoidable library boundaries
- no secrets in client bundles
- structured errors
- predictable loading states
- predictable empty states
- predictable failure states
- accessible keyboard navigation
- semantic HTML
- mobile responsiveness
- query/index awareness
- no obvious N+1 data loading

Shared business logic should not be duplicated between UI and workers.

UI Quality

Every finished screen must include:

- loading state
- empty state
- error state
- realistic long-text handling
- responsive behavior
- keyboard behavior where applicable
- accessibility labels

- source/evidence visibility where relevant

Do not declare a page complete merely because the happy path renders.

Research Integrity

Each research operation must preserve:

- source
- URL
- retrieval time
- parser version
- extractor version
- records discovered
- claims created
- records modified
- confidence changes
- errors

No silent historical edits.

Working Mode

Work on one build slice at a time.

At the beginning of each slice provide:

- SLICE
- Goal:
- Dependencies:
- Schema changes:
- Routes/components:
- Risks:
- Acceptance tests:

Then implement it.

At the end provide:

- BUILD REPORT
- Completed:
- Files created:
- Files changed:
- Migrations:
- Tests:
- Security/RLS verification:
- UI verification:
- Known limitations:
- Acceptance gate result:

Do not move into the next slice unless the current slice passes its acceptance gate.

If a requested shortcut would violate the architecture, explain why and implement the architecture-safe alternative.

The objective is not maximum code generation speed.

The objective is to build Cannagraph without accumulating architectural debt that compromises provenance, identity resolution, evidence integrity, or future internet-scale research.

Cannagraph Acceptance Quality Gate

Do not mark this implementation complete until every applicable gate passes.

Return each item as: PASS, FAIL, or NOT APPLICABLE.

A failure blocks acceptance.

Gate 1 — Architecture Integrity

- ☐ PostgreSQL remains the canonical data store.
- ☐ No duplicate domain system was introduced.
- ☐ Existing domain abstractions were extended instead of bypassed.
- ☐ No important canonical fact exists only inside generated prose or JSON blobs.
- ☐ Names are not being used as entity identity.
- ☐ No ambiguous identity is automatically merged.
- ☐ Conflicting evidence can still coexist.
- ☐ No model-generated statement can become verified without source evidence.

Gate result: _____

Gate 2 — Database Integrity

- ☐ Every schema change exists in a checked-in migration.
- ☐ Migration applies successfully from a clean database.
- ☐ Migration applies successfully against the current development schema.
- ☐ Required foreign keys exist.
- ☐ Required uniqueness constraints exist.
- ☐ Required CHECK constraints exist.
- ☐ Important query paths have appropriate indexes.
- ☐ No destructive migration occurs without explicit justification.
- ☐ Unknown data remains NULL/unknown rather than fabricated.
- ☐ Seed data follows the same constraints as production data.

Gate result: _____

Gate 3 — Provenance Integrity

For any feature creating research data:

- ☐ Source is preserved.
- ☐ Source URL is preserved.
- ☐ Retrieval timestamp is preserved.
- ☐ Source type/evidence class is preserved.
- ☐ Extracted claims reference underlying evidence.

- ☐ Parser/extractor version is traceable.
- ☐ Supporting and contradicting evidence can coexist.
- ☐ Automated operations emit audit events.
- ☐ Important record changes are traceable.
- ☐ Source-copy detection does not count derivative copies as independent confirmation.

Gate result: _____

Gate 4 — Cannabis Domain Integrity

- ☐ Cultivar, cut, phenotype, line, population, accession, grow and batch are not incorrectly collapsed into one universal “strain” record.
- ☐ Mother/father direction is not invented.
- ☐ Same-name records can remain distinct.
- ☐ Clone-only status requires evidence.
- ☐ Lineage is stored as relationships rather than display strings.
- ☐ Alternative lineage accounts can coexist.
- ☐ Chemistry measurements attach to batches/tests.
- ☐ Sensory observations retain their reporting scope.
- ☐ Historical folklore is labeled according to evidence strength.
- ☐ Research completeness represents documentation quality rather than “probability the strain is real.”

Gate result: _____

Gate 5 — Authorization & Security

- ☐ Public access is intentionally defined.
- ☐ Free-user access is intentionally defined.
- ☐ Pro entitlement checks are centralized.
- ☐ Researcher permissions are separate from subscription status.
- ☐ Admin permissions are separate from subscription status.
- ☐ RLS exists for affected tables.
- ☐ Browser code cannot access service-role credentials.
- ☐ Server actions/routes validate input.
- ☐ Client-provided authorization state is never trusted.
- ☐ Unauthorized access was tested, not merely assumed.

Gate result: _____

Gate 6 — Research Worker Safety

Where applicable:

- ☐ Source access policy is checked before retrieval.
- ☐ Blocked/manual-only sources are not automatically fetched.
- ☐ Rate limits are respected.
- ☐ Jobs are idempotent.

- ☐ Duplicate delivery does not duplicate canonical records.
- ☐ Retry policy exists.
- ☐ Permanent failures become visible.
- ☐ Infinite recursive discovery is prevented.
- ☐ Per-source/domain research budgets exist.
- ☐ Research failures never silently create verified records.

Gate result: _____

Gate 7 — Search & Graph Correctness

Where applicable:

- ☐ Aliases resolve without collapsing distinct identities.
- ☐ Lineage traversal excludes rejected edges unless explicitly requested.
- ☐ Alternative/disputed lineage remains visible where relevant.
- ☐ Graph expansion is bounded.
- ☐ Large lineage graphs load incrementally.
- ☐ Descendant and ancestor queries were tested with cycles/invalid data protection.
- ☐ Search results preserve entity type.
- ☐ Evidence-sensitive filters return correct results.
- ☐ Query plans were inspected for important high-volume paths.

Gate result: _____

Gate 8 — UI Quality

- ☐ Screen visually matches the premium Cannagraph direction.
- ☐ No stereotypical cannabis/dispensary styling was introduced.
- ☐ Desktop layout is polished.
- ☐ Tablet layout is usable.
- ☐ Mobile layout is usable.
- ☐ Loading state exists.
- ☐ Empty state exists.
- ☐ Error state exists.
- ☐ Long names/text do not break layout.
- ☐ Evidence/confidence information remains understandable.
- ☐ Primary actions are obvious.
- ☐ Login friction has not been added to public research unnecessarily.
- ☐ UI uses real application data rather than fake final-state JSON.

Gate result: _____

Gate 9 — Accessibility

- ☐ Keyboard navigation works.
- ☐ Focus states are visible.
- ☐ Form fields have labels.

- ☐ Icon-only controls have accessible names.
- ☐ Interactive graph controls have accessible alternatives where practical.
- ☐ Color is not the only signal for confidence/status.
- ☐ Semantic headings are correct.
- ☐ Dialogs/drawers manage focus correctly.

Gate result: _____

Gate 10 — Performance

- ☐ No obvious N+1 queries.
- ☐ Server/client boundaries are intentional.
- ☐ Expensive data is not shipped unnecessarily to the browser.
- ☐ Graph queries have bounded depth.
- ☐ Public pages do not require unnecessary authenticated requests.
- ☐ Appropriate caching/revalidation strategy exists.
- ☐ Large tables paginate or virtualize.
- ☐ Source documents are not loaded when only metadata is required.
- ☐ Important database queries have acceptable execution plans.

Gate result: _____

Gate 11 — Code Quality

- ☐ TypeScript passes in strict mode.
- ☐ Lint passes.
- ☐ Production build passes.
- ☐ Automated tests pass.
- ☐ Boundary inputs are validated.
- ☐ No unexplained any.
- ☐ No dead code from abandoned implementation attempts.
- ☐ No acceptance-critical TODOs.
- ☐ Business rules are not duplicated unnecessarily.
- ☐ Components remain reasonably scoped.

Gate result: _____

Gate 12 — Regression Safety

- ☐ Existing routes still work.
- ☐ Existing migrations still apply.
- ☐ Existing RLS behavior still works.
- ☐ Existing records remain valid.
- ☐ Existing public IDs remain stable.
- ☐ Existing lineage/evidence relationships were not silently altered.
- ☐ Previously completed acceptance tests still pass.

Gate result: _____

Gate result: _____

- ☐ Stable public IDs and activity events can support future API and feed access.
- ☐ Canonical research acceptance remains appropriately permissioned.
- ☐ RLS protects user and organization-owned content.
- ☐ Privacy controls exist for sensitive grow data.
- ☐ Reputation does not determine factual truth.
- ☐ Organization ownership cannot suppress conflicting historical evidence.
- ☐ Moderation actions are auditable.
- ☐ Claim challenges preserve the original claim and competing evidence.
- ☐ Community sensory observations remain observer/source scoped.
- ☐ Community chemistry remains batch/test-specific.
- ☐ Grow journals use structured events and remain scoped to the actual grow/batch.
- ☐ Repeated copies of the same originating source do not count as independent evidence.
- ☐ Community claims retain source, author, organization, and verification provenance.
- ☐ Community submissions cannot directly overwrite canonical facts.
- ☐ Verification is independent from subscription level and researcher/admin authorization.
- ☐ Organizations are distinct from individual users and from breeder/history records where appropriate.
- ☐ Community records reference canonical entities rather than duplicating them.

Gate 13 — Community & Organization Integrity

Required Final Verification

Commands Executed

List the actual typecheck, lint, tests, production build, and database migration verification commands and their results.

Database Verification

Describe:

- migrations added
- constraints added
- indexes added
- RLS policies added/changed
- data migrations/backfills performed

Manual Verification

List the exact screens and workflows manually checked.

Known Limitations

Explicitly list anything unfinished or provisional. Do not hide incomplete work.

Final Decision

Return exactly one:

Acceptance Result

ACCEPTANCE GATE: PASS

or

ACCEPTANCE GATE: FAIL

If FAIL, list the blocking defects before proposing additional work.

Correct hierarchy: canonical knowledge graph <- claims/evidence <- research review <- community contributions <- users/growers/breeders/organizations. Internet discovery feeds the same review pipeline from the opposite direction.

Primary risks to test: parallel identity systems, duplicated cultivar records, untraceable community claims, verification abuse, organization ownership conflicts, privacy leakage, evidence inflation, and uncontrolled social features.

The review must list existing systems affected, schema additions required now, schema additions safely deferred, the users/organizations/breeders/cultivars/cuts/grows/batches/journals/submissions/claims/evidence/source relationship map, RLS model, provenance model, migration plan, risks, and exact Slice 1 recommendations.

Before implementing community UI or broad social features, perform an architecture impact review and stop.

Architecture Impact Review Requirement

Vibe-Coding Operating Rule

Never prompt the coding agent with “build Cannagraph.” Give it the master prompt once, then issue a bounded slice command such as “Implement Slice 1 only. Stop after running the acceptance quality gate.” This keeps the agent from racing ahead and producing half-integrated systems.